TP bandit à N bras : apprentissage par renforcement

Implémentation du TP 1 - bandit manchot : un bandit à 10 bras et trois stratégies (ε-greedy, initialisation optimiste, UCB), comparées en stationnaire puis en non-stationnaire (bonus).

Membres du groupe : AGANZE LWABOSHI Moïse, Gédéon MUTIKANGA, Jonas AMOZIGH.

Répartition du travail

Chacun a pris en charge des questions faciles, des questions de code et des questions d'interprétation. Pour chaque question, la même personne a fait le code, les graphiques et la rédaction de la réponse dans le rapport.

Exercice	Questions	Réalisé par
1. Le bandit manchot	1.1-1.2 (classe `Bandit`, `play`)	AGANZE LWABOSHI Moïse
	1.3-1.4 (histogrammes 1 et 3 bandits)	Gédéon MUTIKANGA
2. Le Ban10	2.1-2.3	Jonas AMOZIGH
3. ε-greedy	3.1-3.4 (constructeur, Q/N, `get_action`, `explore`)	Gédéon MUTIKANGA
	3.5-3.8 (`_greedy_action`, `_random_action`, choix, `reward`)	AGANZE LWABOSHI Moïse
	3.9-3.11 (boucle de jeu et affichage)	AGANZE LWABOSHI Moïse
4. Graphiques	4.1 (récompense d'un seul joueur)	Jonas AMOZIGH
	4.2-4.4 (2000 joueurs étape par étape, moyennes, % optimal)	AGANZE LWABOSHI Moïse
	4.5-4.6 (comparaison des ε et interprétation)	Gédéon MUTIKANGA
5. Initialisation optimiste	5.1-5.4 (dont l'extension α = 0.1)	Jonas AMOZIGH
6. UCB	6.1-6.4	AGANZE LWABOSHI Moïse
7. Non-stationnaire (bonus)	7.1-7.2 (`MovingBandit`, `MovingBan10`)	AGANZE LWABOSHI Moïse
	7.3 (valeurs de ε) et extension α constant	Jonas AMOZIGH
	7.4 (optimiste et UCB)	Gédéon MUTIKANGA
Rapport	assemblage, section « Reproduire les résultats »	AGANZE LWABOSHI Moïse

Exercice	Questions	Réalisé par
	relecture croisée de toutes les réponses	les trois membres

Structure du projet

Fichier	Contenu
`bandit.py`	Exercices 1, 2, 7.1-7.2 : `Bandit`, `Ban10`, `MovingBandit`, `MovingBan10`.
`players.py`	Exercices 3, 5, 6 : `GreedyPlayer`, `OptimistGreedyPlayer`, `UCBGreedyPlayer`.
`experiments.py`	Exercice 4 : `run_experiment` (2000 problèmes joués étape par étape) et tracé des courbes.
`main.py`	Une commande par exercice ; les figures sont écrites dans `figures/`.

Dépendances : Python 3, `numpy`, `matplotlib` (`pip install numpy matplotlib`).

Reproduire les résultats

```
python main.py all          # tous les exercices (~2 min), graine 42
python main.py ex4          # un exercice seul
python main.py ex3 --verbose          # affiche les 1000 coups
python main.py ex6 --c 1 --runs 500   # options : --runs --steps --eps --c --seed
```

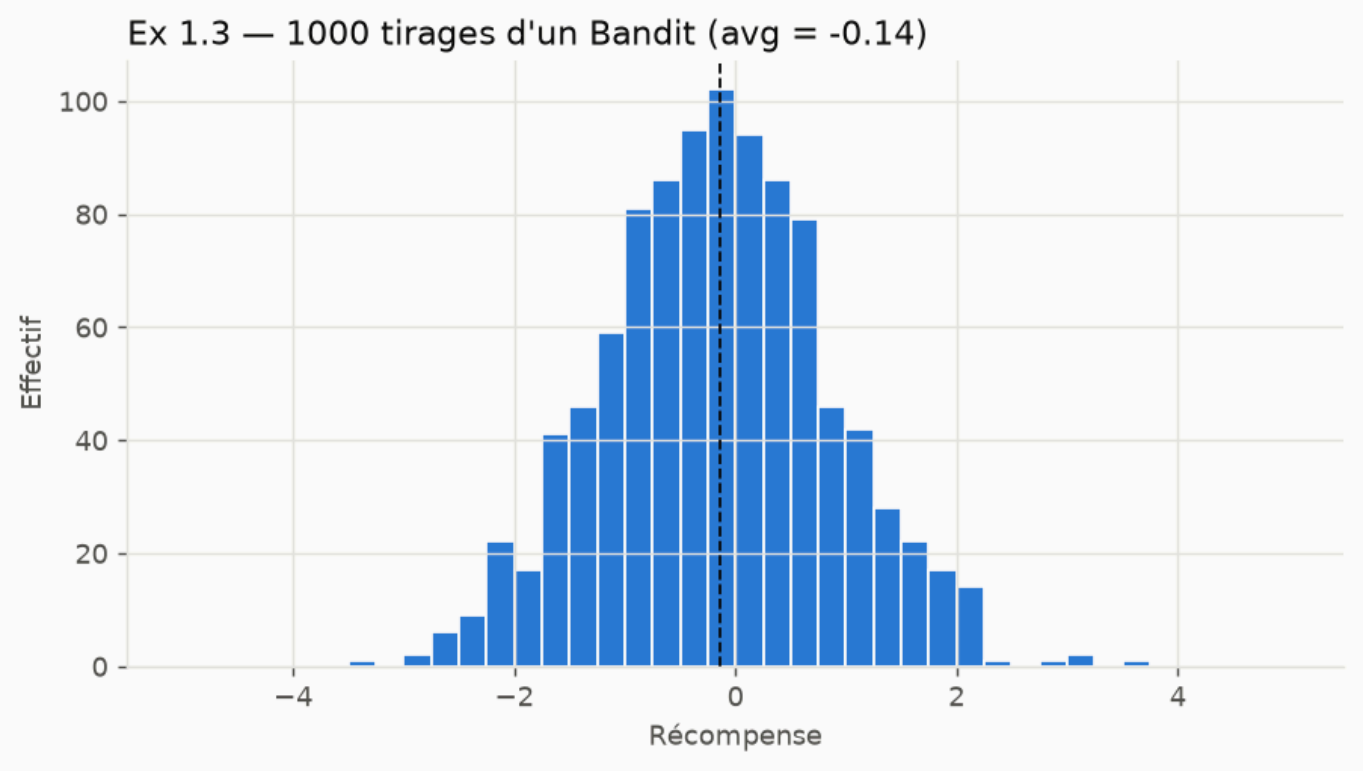
Tous les chiffres ci-dessous viennent de `python main.py all` (graine 42, 2000 problèmes × 1000 coups). « Fin » = moyenne sur les 100 derniers coups. La graine rend les résultats reproductibles. Lancer un exercice seul donne des chiffres légèrement différents, car la séquence aléatoire n'est plus la même.

Exercice 1 : le bandit manchot

Réalisé par : 1.1-1.2 AGANZE LWABOSHI Moïse ; 1.3-1.4 Gédéon MUTIKANGA.

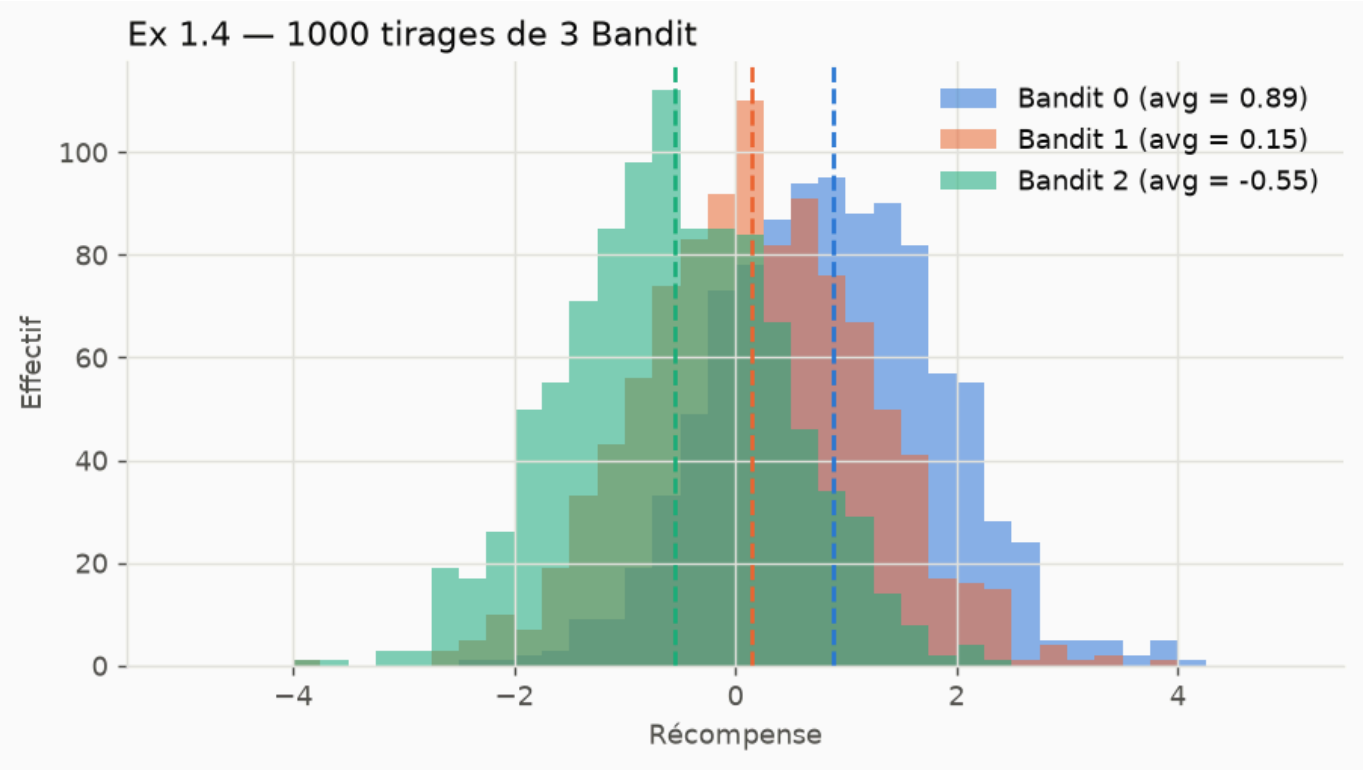
1.1-1.2 `Bandit.__init__` tire `avg ~ N(0, 1)` ; `play()` renvoie un tirage `N(avg, 1)`.

1.3 1000 tirages d'un même bandit : l'histogramme a la forme en cloche d'une gaussienne, centrée sur `avg` (ligne pointillée), d'écart-type ≈ 1.



1.4 Trois bandits : trois cloches de même largeur mais de pics différents, chacun centré sur son propre avg. Les moyennes empiriques retrouvent bien les vraies moyennes :

	avg	**moyenne de 1000 tirages**
Bandit 0	0.889	0.871
Bandit 1	0.153	0.189
Bandit 2	−0.550	−0.573



Les distributions se recouvrent largement : un seul tirage ne suffit pas à savoir quel bras est le meilleur. C'est tout l'enjeu du problème.

Exercice 2 : le Ban10

Réalisé par : Jonas AMOZIGH.

2.1 `Ban10.__init__` crée une liste de 10 `Bandit`. **2.2** `best_action` est l'indice du bandit de plus grand `avg`. **2.3** `play(action)` renvoie `self.bandits[action].play()`.

Exemple (`python main.py ex2`) :

```
bras 0 : avg = +0.998
...
bras 6 : avg = +1.183
...
bras 9 : avg = +1.786  <- meilleur
best_action = 9
```

Exercice 3 : algorithme ε-greedy

Réalisé par : 3.1-3.4 Gédéon MUTIKANGA ; 3.5-3.11 AGANZE LWABOSHI Moïse.

3.1-3.2 `GreedyPlayer(n, eps)` stocke `n`, `eps`, et les tableaux `action_values` (Q) et `eval_count` (N), initialisés à 0.

3.3-3.7 `get_action()` tire `random.random()`, pose `explore = valeur < eps`, puis appelle `_random_action()` (bras uniforme) si `explore` vaut True, sinon `_greedy_action()` (argmax de Q, égalités départagées au hasard).

3.8 `reward(action, r)` met à jour par moyenne incrémentale :

```
N(a) <- N(a) + 1
Q(a) <- Q(a) + (r - Q(a)) / N(a)
```

Un paramètre optionnel `alpha` remplace `1/N(a)` par un pas constant. Il n'est utilisé qu'aux exercices 5 (comparaison) et 7.

3.9-3.11 Boucle de 1000 itérations `get_action → Ban10.play → reward`, qui affiche la récompense et si le choix était optimal (`python main.py ex3`) :

```
coup    0 : bras 8  récompense -1.246  -
coup    1 : bras 9  récompense -0.652  -
coup    2 : bras 7  récompense -1.809  -
coup    3 : bras 2  récompense -1.080  -
coup    4 : bras 3  récompense +0.581  optimal
coup    5 : bras 3  récompense +1.388  optimal
```

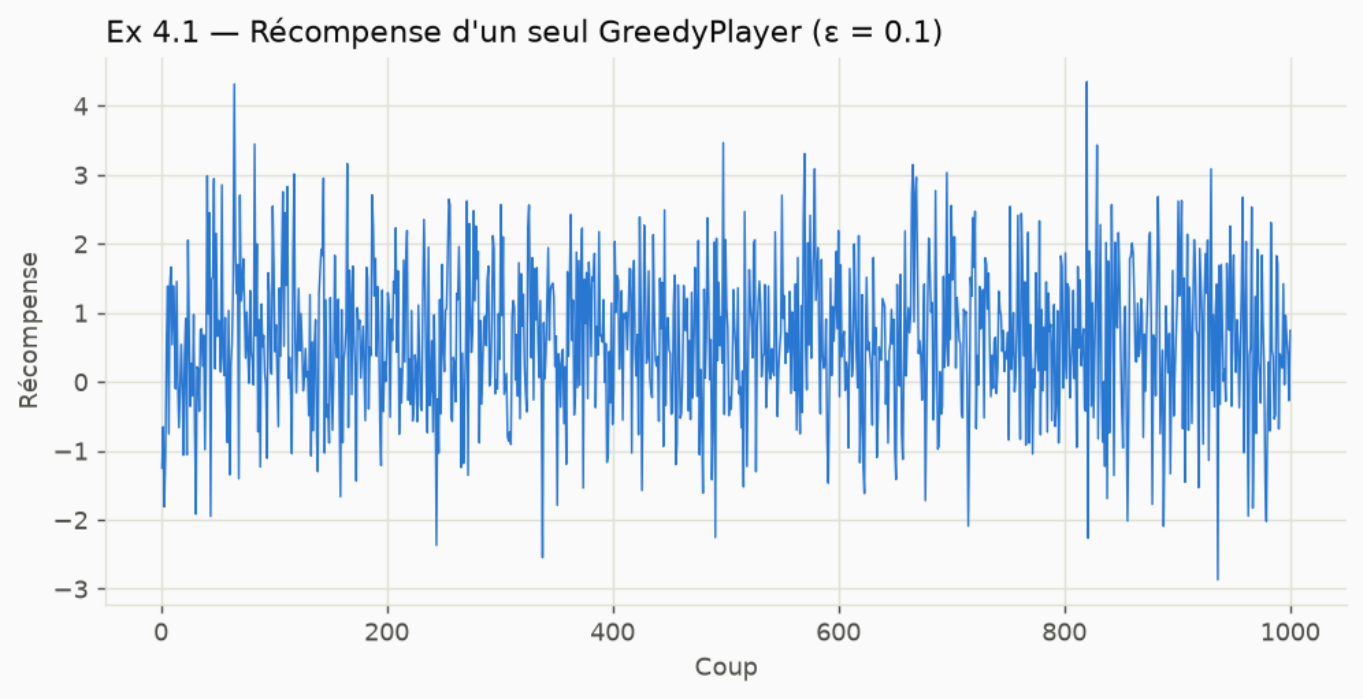
```
...
Choix optimal sur 867/1000 coups.
```

On voit le comportement greedy au démarrage : tout est à 0, et un bras qui donne une récompense négative passe sous 0. Le joueur essaie donc d'autres bras jusqu'à en trouver un positif (ici le bras 3, le meilleur), puis il s'y tient.

Exercice 4 : graphiques simples

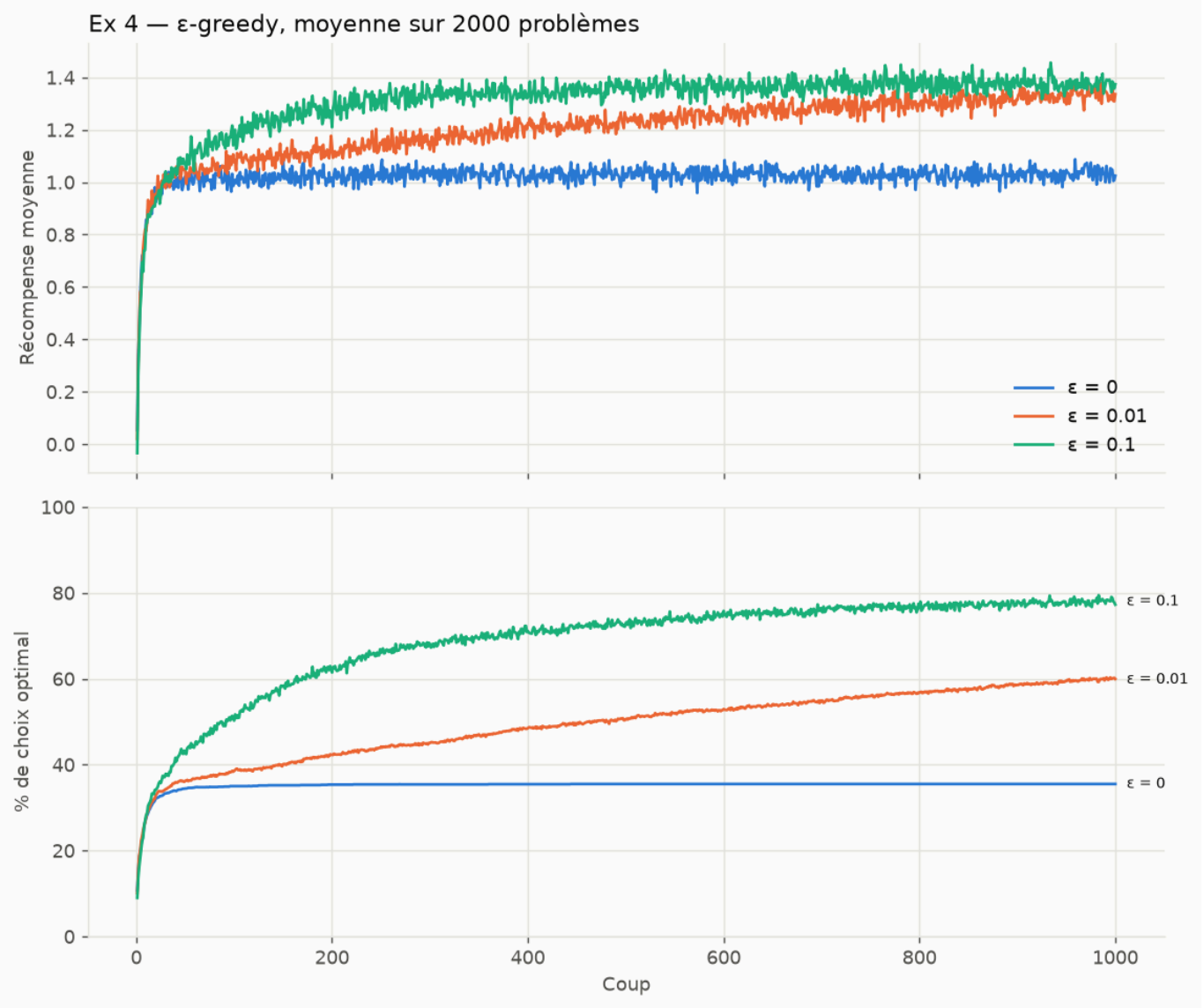
Réalisé par : 4.1 Jonas AMOZIGH ; 4.2-4.4 AGANZE LWABOSHI Moïse ; 4.5-4.6 Gédéon MUTIKANGA.

4.1 La récompense d'un seul joueur est très bruitée : chaque tirage a une variance de 1, du même ordre que les écarts entre bras. On ne voit aucune tendance.



4.2-4.4 `run_experiment` crée 2000 couples (Ban10, joueur) et les fait jouer **étape par étape** : les 2000 jouent leur coup 1, puis leur coup 2, etc. À chaque coup on enregistre la récompense moyenne et le pourcentage de joueurs ayant choisi `best_action`.

4.5 ε = 0, 0.01, 0.1 :



ε	récompense (fin)	% optimal (fin)
0	1.03	35.5 %
0.01	1.33	59.4 %
0.1	1.38	77.8 %

4.6 Interprétation.

Comportement initial. Au coup 1, tous les Q valent 0 et le choix est aléatoire : la récompense moyenne vaut ≈ 0 et 10 % des choix sont optimaux. Les trois courbes montent ensuite très vite pendant ~20 coups, car le joueur abandonne les bras qui donnent du négatif. ε = 0 est même un peu devant au tout début, puisqu'il ne « gaspille » aucun coup en exploration.

Comportement asymptotique.

- **ε = 0** plafonne dès le coup ~50, à 1.03 et 35 % de choix optimaux. Il se fige sur le premier bras dont l'estimation reste positive. Les autres bras, qui n'ont jamais été essayés ou ont été sous-estimés sur un mauvais tirage, ne sont jamais réévalués.
- **ε = 0.1** explore beaucoup : il trouve le meilleur bras plus vite et domine sur 1000 coups. Mais il continue de jouer au hasard 10 % du temps, donc il ne peut pas dépasser 1 − ε + ε/10 = **91 %** de choix

optimaux.

- **ε = 0.01** apprend plus lentement (sa courbe monte encore au coup 1000) mais son plafond théorique est de **99.1 %**. Sur un horizon assez long, il finirait par dépasser ε = 0.1.

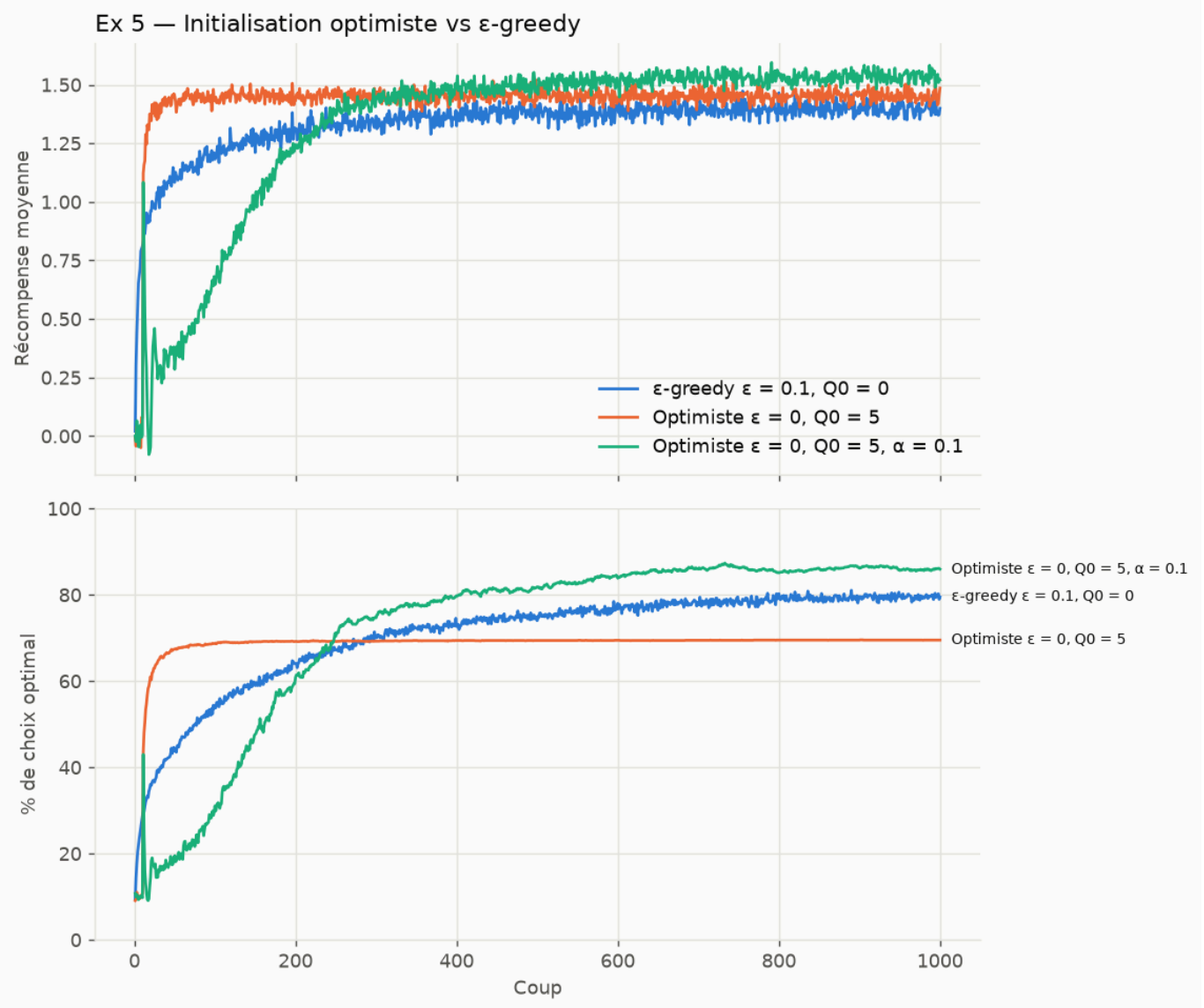
La récompense ne peut pas dépasser l'espérance du meilleur bras, en moyenne E[max de 10 N(0,1)] ≈ 1.54. C'est le compromis exploration / exploitation : explorer coûte à court terme mais évite de se bloquer sur un mauvais bras.

Exercice 5 : initialisation optimiste

Réalisé par : Jonas AMOZIGH.

5.1-5.2 `OptimistGreedyPlayer` hérite de `GreedyPlayer`. Son constructeur met Q à 5 et ε = 0 par défaut : c'est l'optimisme seul qui fait explorer.

5.3 `run_experiment` crée **un** Ban10 par problème, puis donne à chaque stratégie une copie (`deepcopy`) de ce même Ban10. Les deux joueurs sont donc évalués sur exactement les mêmes 2000 problèmes. La copie évite que deux joueurs fassent avancer le même objet (nécessaire pour le `MovingBan10` de l'exercice 7).



Joueur	récompense (fin)	% optimal (fin)
ε-greedy ε = 0.1, Q0 = 0	1.40	79.5 %
Optimiste ε = 0, Q0 = 5 (moyenne empirique)	1.45	69.5 %
Optimiste ε = 0, Q0 = 5, α = 0.1 (extension)	**1.54**	**86.1 %**

5.4 Interprétation.

Comportement initial. Q0 = 5 est bien au-dessus de toute récompense réaliste. Chaque tirage « déçoit » donc et fait baisser l'estimation du bras joué sous celle des bras pas encore essayés. Le joueur essaie ainsi systématiquement les 10 bras pendant les 10 premiers coups : la récompense reste ≈ 0 et 10 % des choix sont optimaux. Ensuite, il exploite les estimations obtenues et monte beaucoup plus vite qu'ε-greedy (≈ 69 % de choix optimaux dès le coup 50).

Comportement asymptotique. Avec la moyenne empirique, le premier pas vaut 1/N = 1/1 : **Q0 est entièrement oublié dès le premier tirage de chaque bras** (Q devient la première récompense). Le joueur optimiste revient donc à « essayer chaque bras une fois, puis greedy pur ». Il ne réexplore plus jamais et se fige à 69.5 % de choix optimaux, avec une courbe parfaitement plate. Sa récompense reste supérieure à celle d'ε-greedy (1.45 contre 1.40) : quand il se trompe, c'est en général sur un bras proche du meilleur, et il ne perd pas 10 % de ses coups en hasard.

Avec un pas constant α = 0.1 (le réglage de Sutton & Barto), l'optimisme ne s'efface que progressivement : il faut une dizaine de tirages par bras pour que Q descende au niveau réel. L'exploration initiale est donc bien plus longue, d'où un démarrage lent (≈ 250 coups). Elle est aussi plus fiable, et le joueur finit meilleur que tous les autres : 86 % de choix optimaux contre 80 % pour ε = 0.1, et la meilleure récompense (1.54). Limite de la méthode : elle n'explore qu'au début, ce qui la rend inadaptée au non-stationnaire (voir l'exercice 7).

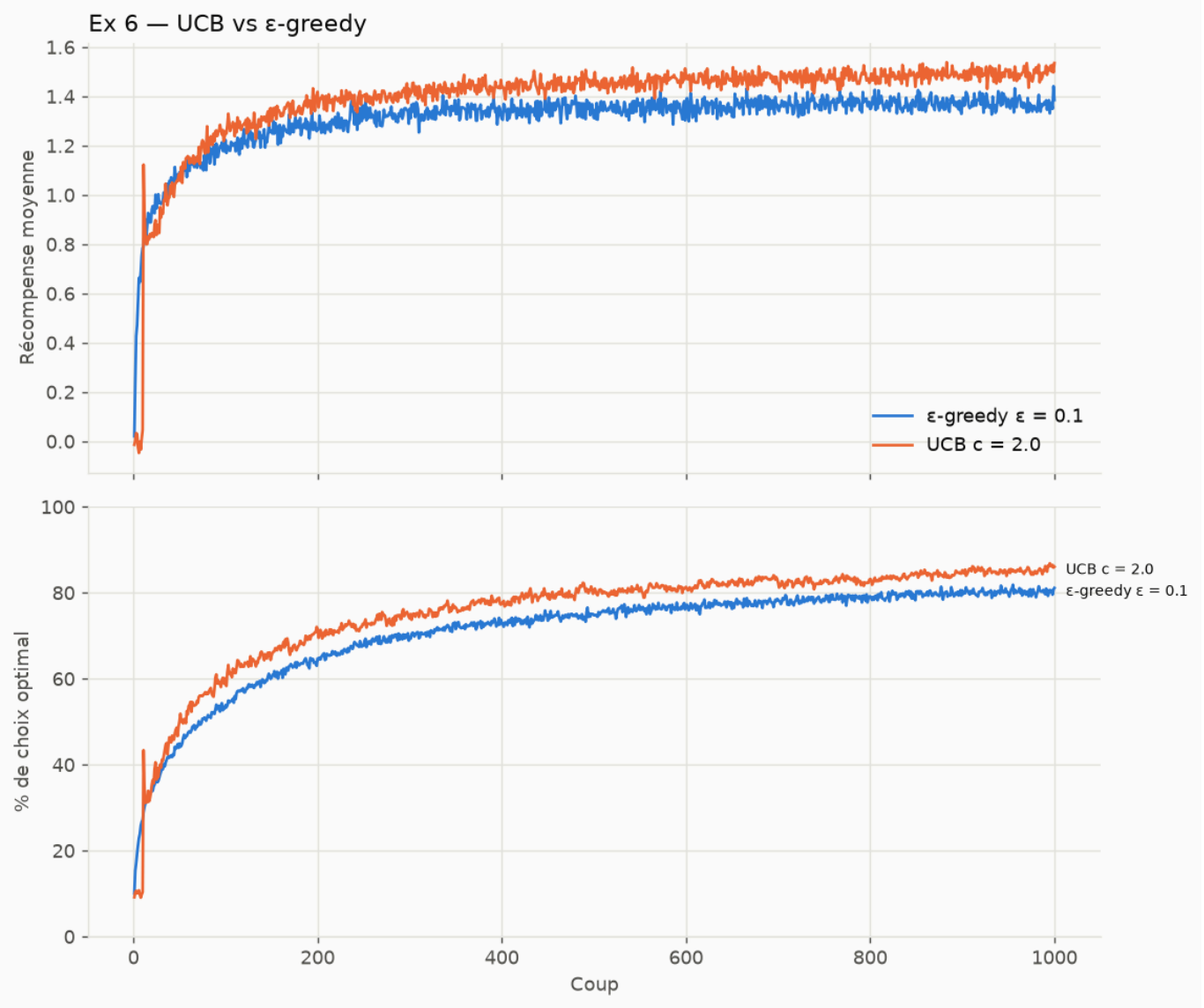
Exercice 6 : sélection Upper-Confidence-Bound

Réalisé par : AGANZE LWABOSHI Moïse.

6.1-6.2 `UCBGreedyPlayer(n, c)` surcharge `_greedy_action` :

```
A_t = argmax_a [ Q(a) + c * sqrt( ln t / N(a) ) ]
```

`t` est le nombre d'appels à `reward` (`self.steps`). Une action jamais essayée (N(a) = 0) est considérée comme maximisante : elle est choisie en priorité. On prend c = 2 et ε = 0.



Joueur	récompense (fin)	% optimal (fin)
ε-greedy ε = 0.1	1.37	80.2 %
UCB c = 2	**1.49**	**85.1 %**

6.4 Interprétation.

Comportement initial. Pendant les 10 premiers coups, UCB essaie chaque bras une fois : la récompense est ≈ 0. Au **coup 11**, la courbe fait un pic net (≈ 42 % de choix optimaux). À ce moment tous les N valent 1, donc le bonus est le même pour tous les bras, et UCB choisit simplement le bras dont le premier tirage était le meilleur, souvent le bon. Au coup 12, ce bras a N = 2 et donc un bonus plus faible que les autres : UCB repart explorer et la courbe redescend. Comme les 2000 joueurs suivent ce même calendrier, le pic est visible sur la moyenne. Ensuite, UCB est légèrement au-dessus d'ε-greedy.

Comportement asymptotique. UCB est nettement meilleur (1.49 contre 1.37). Son exploration est **dirigée** : il ne réessaie que les bras encore incertains (N petit) ou prometteurs (Q élevé). ε-greedy, lui, continue de jouer des bras clairement mauvais 10 % du temps. De plus, le bonus `sqrt(ln t / N)` diminue à mesure que N augmente : UCB explore de moins en moins, et n'est pas bloqué par un plafond comme le 91 % d'ε = 0.1. Contreparties : c doit être réglé, et la méthode se généralise mal au-delà des bandits (grands espaces d'états, non-stationnarité).

Exercice 7 (bonus) : problèmes non-stationnaires

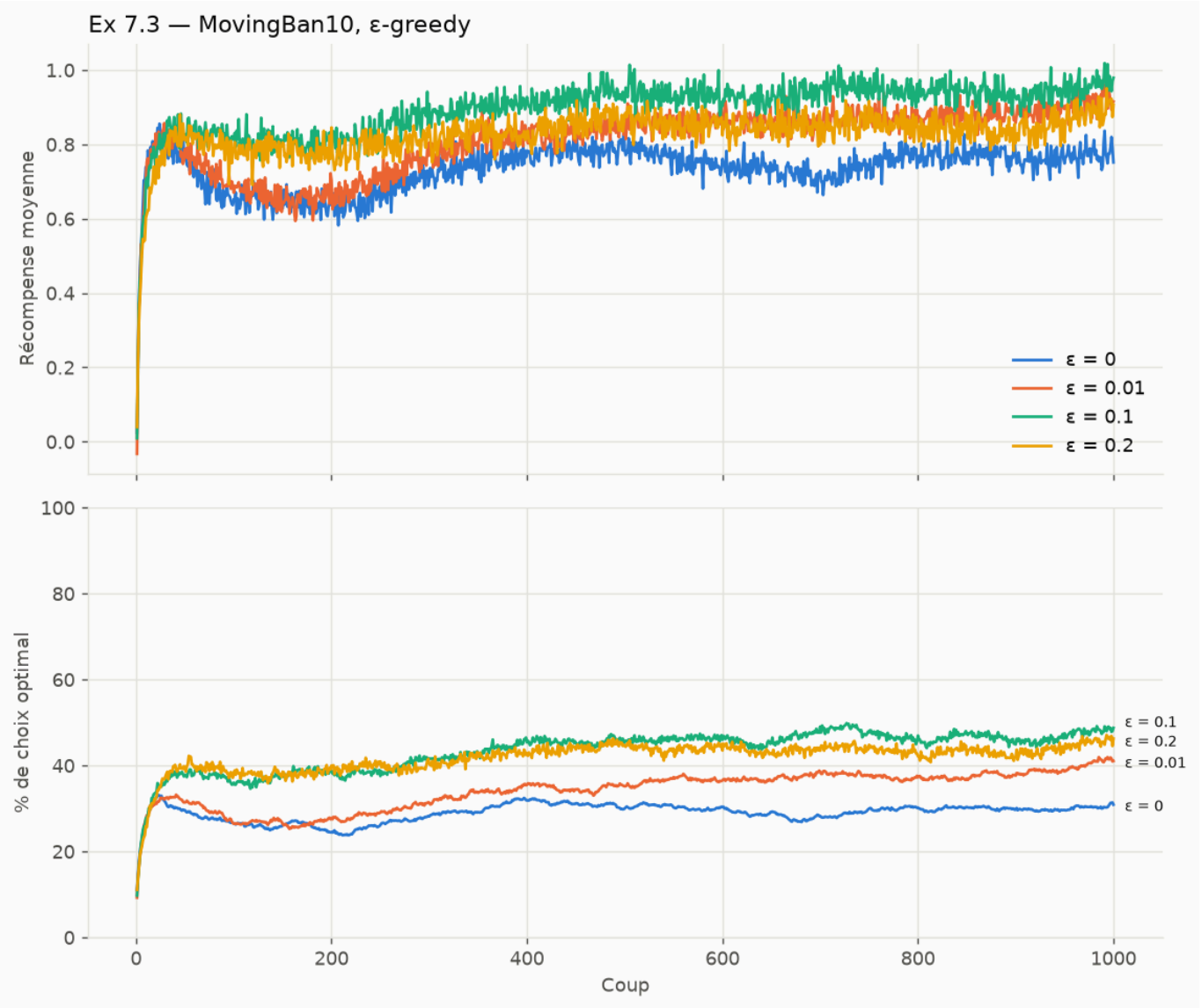
Réalisé par : 7.1-7.2 AGANZE LWABOSHI Moïse ; 7.3 et extension α Jonas AMOZIGH ; 7.4 Gédéon MUTIKANGA.

7.1 `MovingBandit` : `avg(t) = centre + amplitude · sin(2π t / période + phase)`.

- min et max sont deux tirages N(0, 1), triés : ils donnent le centre et l'amplitude ;
- la période est tirée uniformément entre 100 et 500 coups ;
- la phase est aléatoire, pour que les bras ne partent pas tous du même point du sinus ;
- `avg` est mis à jour à chaque `play()`.

7.2 `MovingBan10` instancie des `MovingBandit`. Choix de conception : à chaque coup, **tous** les bras avancent d'un pas (le bras joué via `play`, les autres via `step`). Si seul le bras joué bougeait, un bras délaissé resterait figé. La vraie difficulté disparaîtrait alors : un bras ignoré qui devient le meilleur sans que l'agent le voie. `best_action` est recalculé après chaque coup. Le « choix optimal » est jugé par rapport au meilleur bras **au moment du coup**.

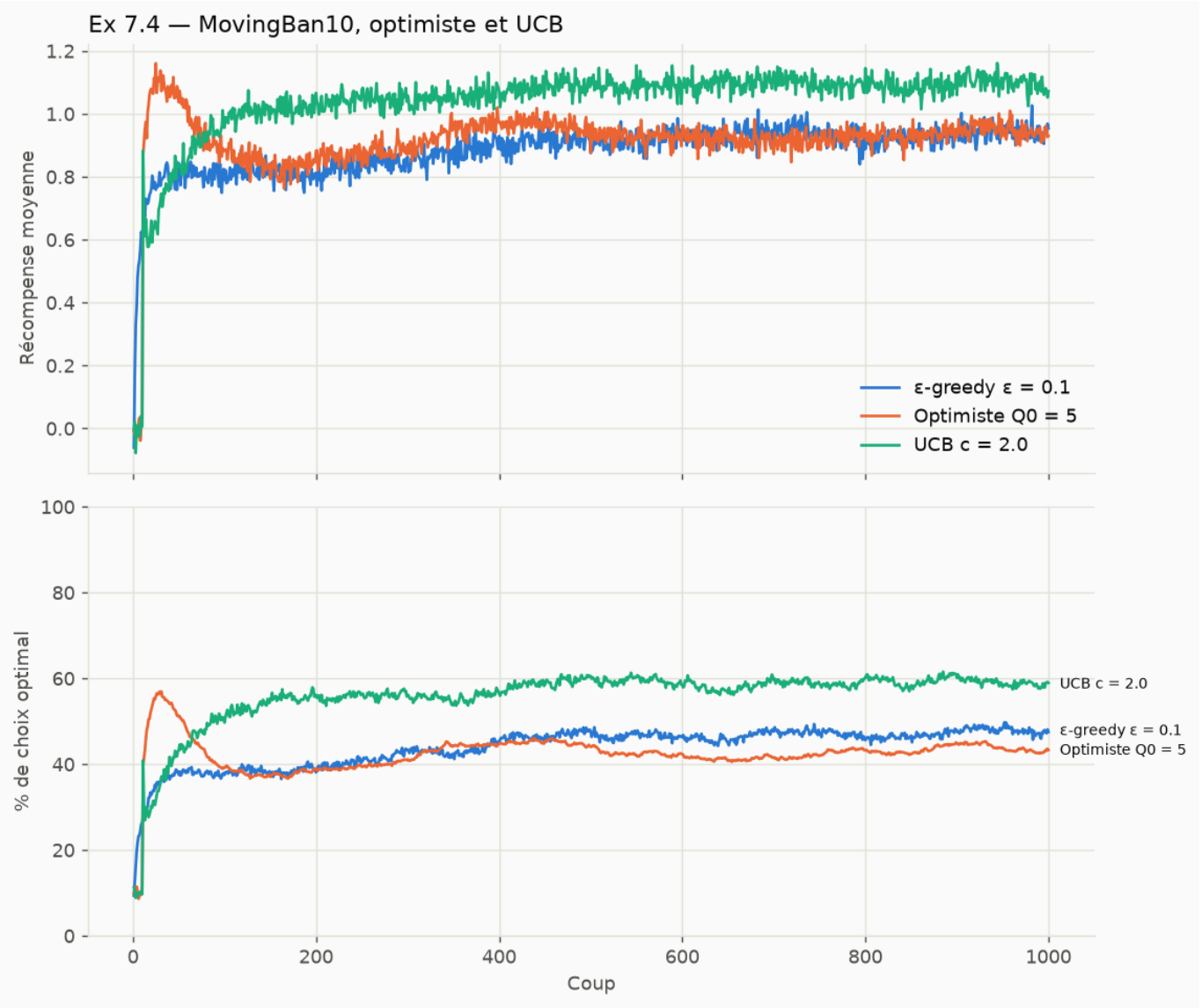
7.3 Différentes valeurs de ε :



ε	récompense (fin)	% optimal (fin)
0	0.77	30.0 %
0.01	0.90	39.5 %
0.1	**0.95**	**46.9 %**
0.2	0.87	44.4 %

Tous les scores chutent par rapport au stationnaire (≈ 47 % de choix optimaux au lieu de 78 %), car le meilleur bras change au fil du temps. ε = 0 souffre le plus : sa récompense **baisse** entre les coups 50 et 200, parce que le bras sur lequel il s'est figé descend sur son sinus et qu'il ne le voit pas. Il faut explorer, mais sans excès : ε = 0.2 fait moins bien que 0.1.

7.4 Optimiste et UCB (moyenne empirique) :

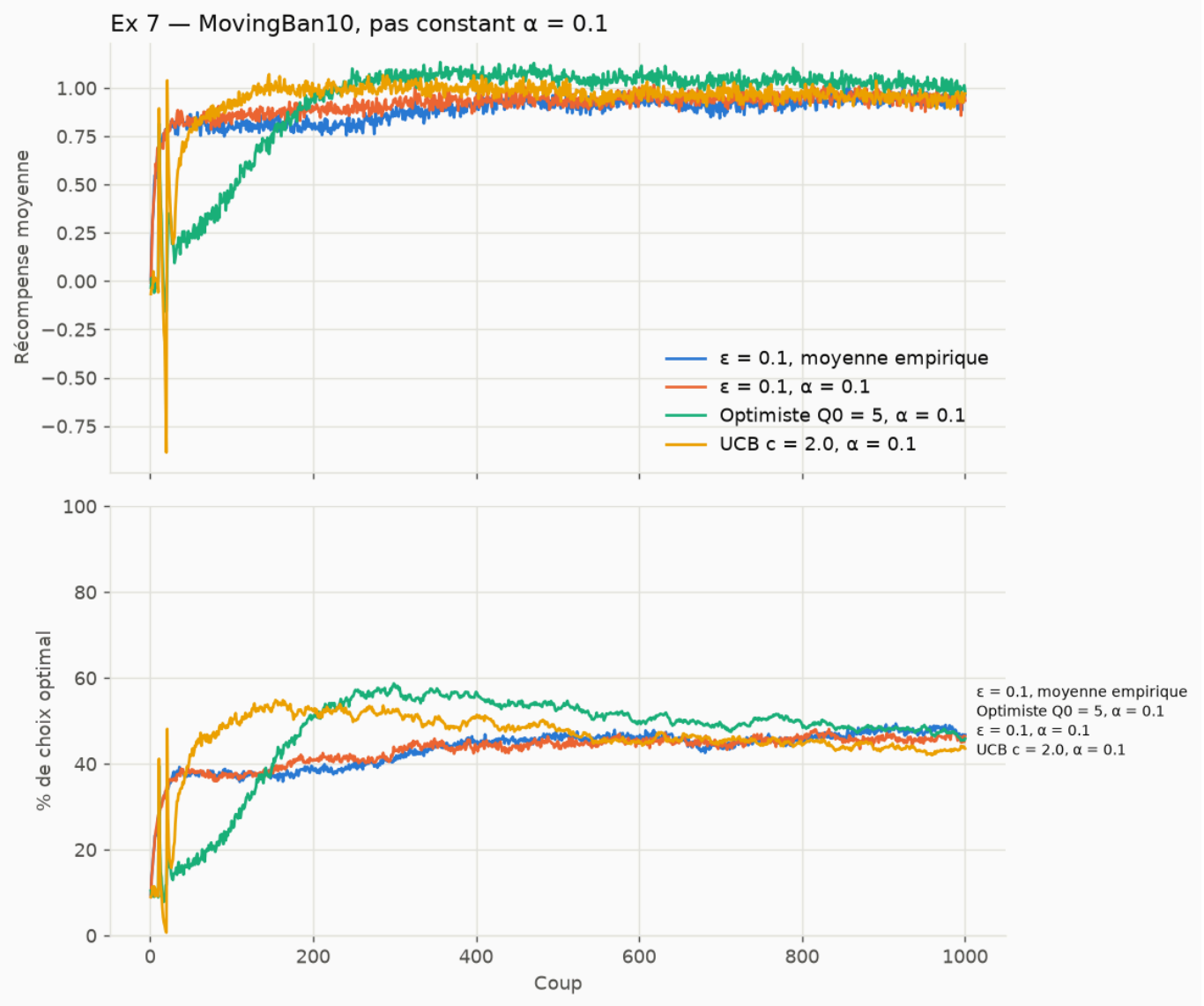


Joueur	récompense (fin)	% optimal (fin)
ε-greedy ε = 0.1	0.94	47.9 %
Optimiste Q0 = 5	0.95	43.9 %

Joueur	récompense (fin)	% optimal (fin)
UCB c = 2	**1.10**	**59.0 %**

- **L'initialisation optimiste** est très bonne au début (pic à ≈ 55 % vers le coup 30). Mais son exploration n'a lieu qu'une fois, au départ. Elle retombe ensuite au niveau d'ε-greedy, puis légèrement en dessous, quand les bras se mettent à bouger.
- **UCB** est le meilleur : le terme `ln t` augmente avec le temps alors que N(a) reste constant pour un bras délaissé. Le bonus de ce bras remonte donc progressivement, et UCB revient régulièrement vérifier les bras qu'il n'a pas joués depuis longtemps.

Extension : pas constant α = 0.1. En non-stationnaire, la théorie recommande un pas constant, qui pondère davantage les récompenses récentes :



Joueur	récompense (fin)	% optimal (fin)
ε = 0.1, moyenne empirique	0.94	47.8 %
ε = 0.1, α = 0.1	0.94	45.9 %
Optimiste Q0 = 5, α = 0.1	**1.01**	47.6 %

Joueur	récompense (fin)	% optimal (fin)
UCB c = 2, α = 0.1	0.95	43.3 %

Le gain est faible, voire négatif, ce qui surprend à première vue. Voici nos hypothèses pour l'expliquer :

- **Pour ε-greedy, le facteur limitant est la fréquence d'exploration, pas la façon de faire la moyenne.** Avec ε = 0.1, chaque bras non-greedy est tiré environ 1 % du temps, soit à peu près un tirage tous les 100 coups. C'est l'ordre de grandeur de la période du sinus : l'estimation est périmée, quelle que soit la façon de la moyenner. Un α constant rend aussi les estimations plus bruitées (≈ 10 derniers tirages seulement).
- **L'optimiste** profite d'α : sa longue exploration initiale (voir l'exercice 5) lui donne de meilleures estimations au départ.
- **UCB avec α est moins bon.** Le bonus repose sur N(a), le nombre total de tirages, alors que Q ne « se souvient » que des ~10 derniers. Le bonus sous-estime donc l'incertitude réelle de l'estimation. On voit aussi un creux marqué vers le coup 20 : après un seul tirage, Q = 0.1·r est presque nul. Le bonus domine alors, et les 2000 joueurs parcourent les bras dans le même ordre de manière synchronisée.

Pour vraiment tirer parti d'α en non-stationnaire, il faudrait l'associer à une exploration qui ne s'éteint pas avec le temps, par exemple UCB avec des compteurs N « oubliés » (*discounted UCB*).